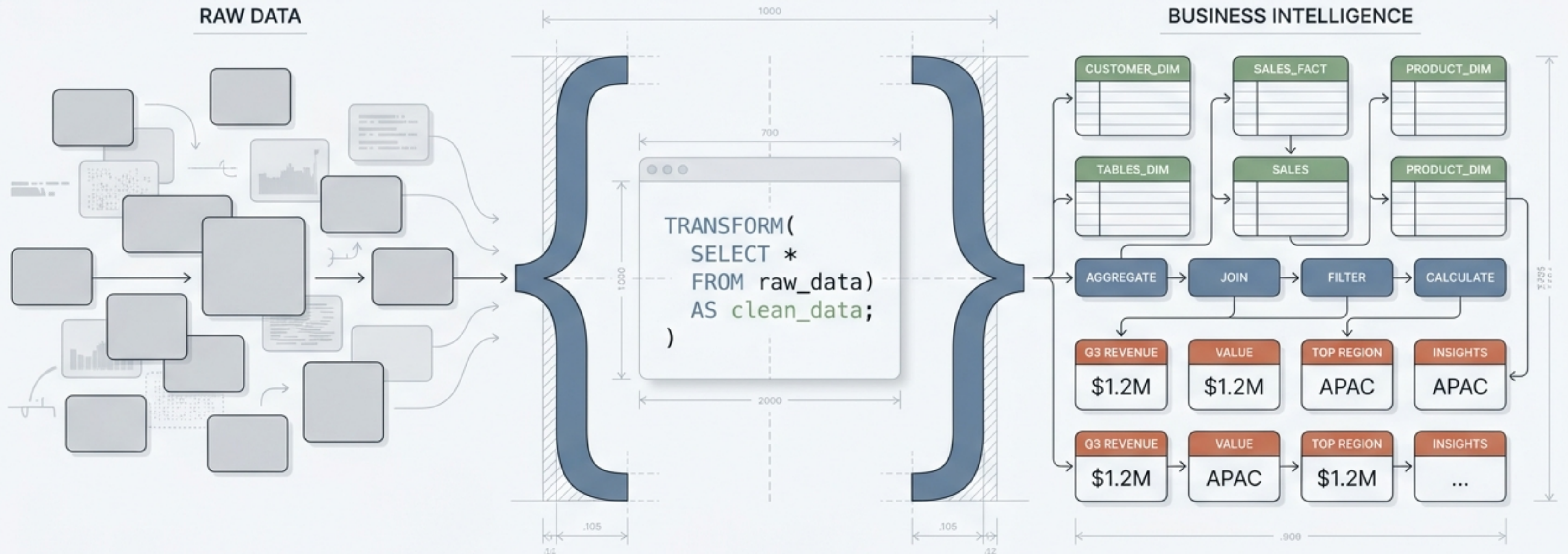


From Raw Data to Business Intelligence

A visual guide to advanced SQL architecture for data science.



The Anatomy of a Database

Columns

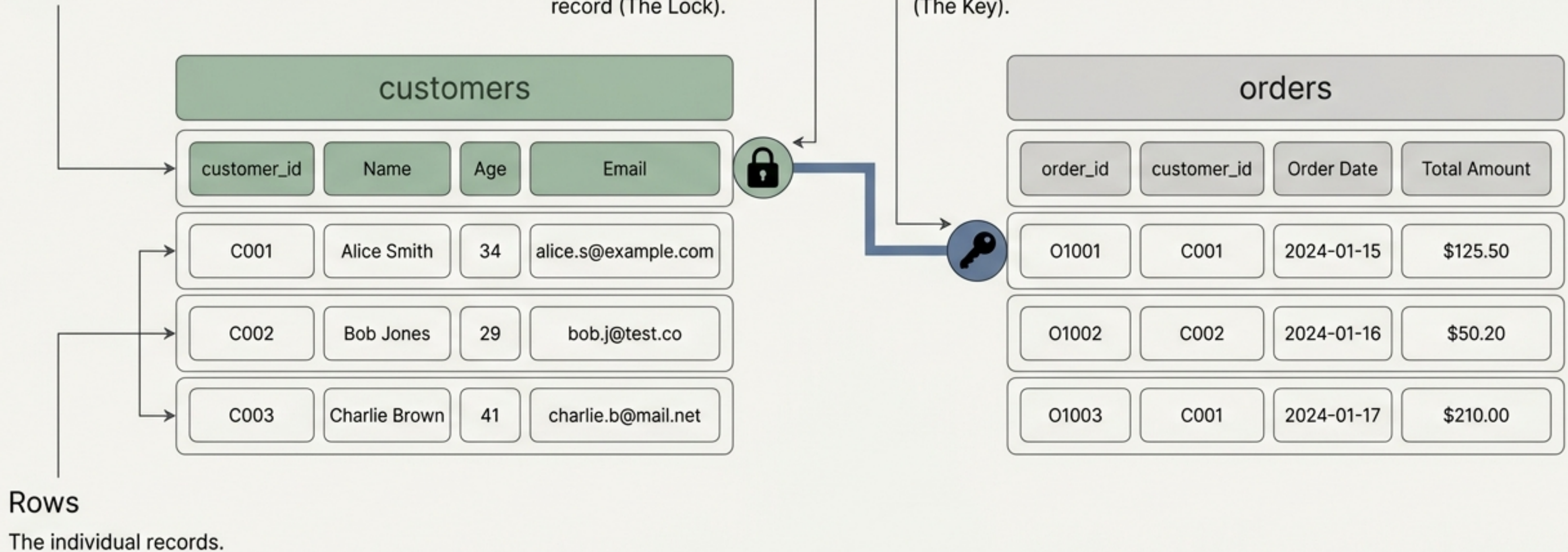
The attributes (e.g., Name, Age).

Primary Key

Uniquely identifies each record (The Lock).

Foreign Key

Connects tables together (The Key).



The Engine Execution Timeline



```
SELECT customer_id, SUM(Total Amount) AS total_spent
FROM orders
WHERE Order Date >= '2024-01-01'
GROUP BY customer_id
HAVING total_spent > 100
ORDER BY total_spent DESC;
```

Pro Tip / Interview Alert:

SQL is written declaratively, but executed sequentially.

Knowing this timeline is the secret to debugging complex queries in production.

Anatomy of a Query

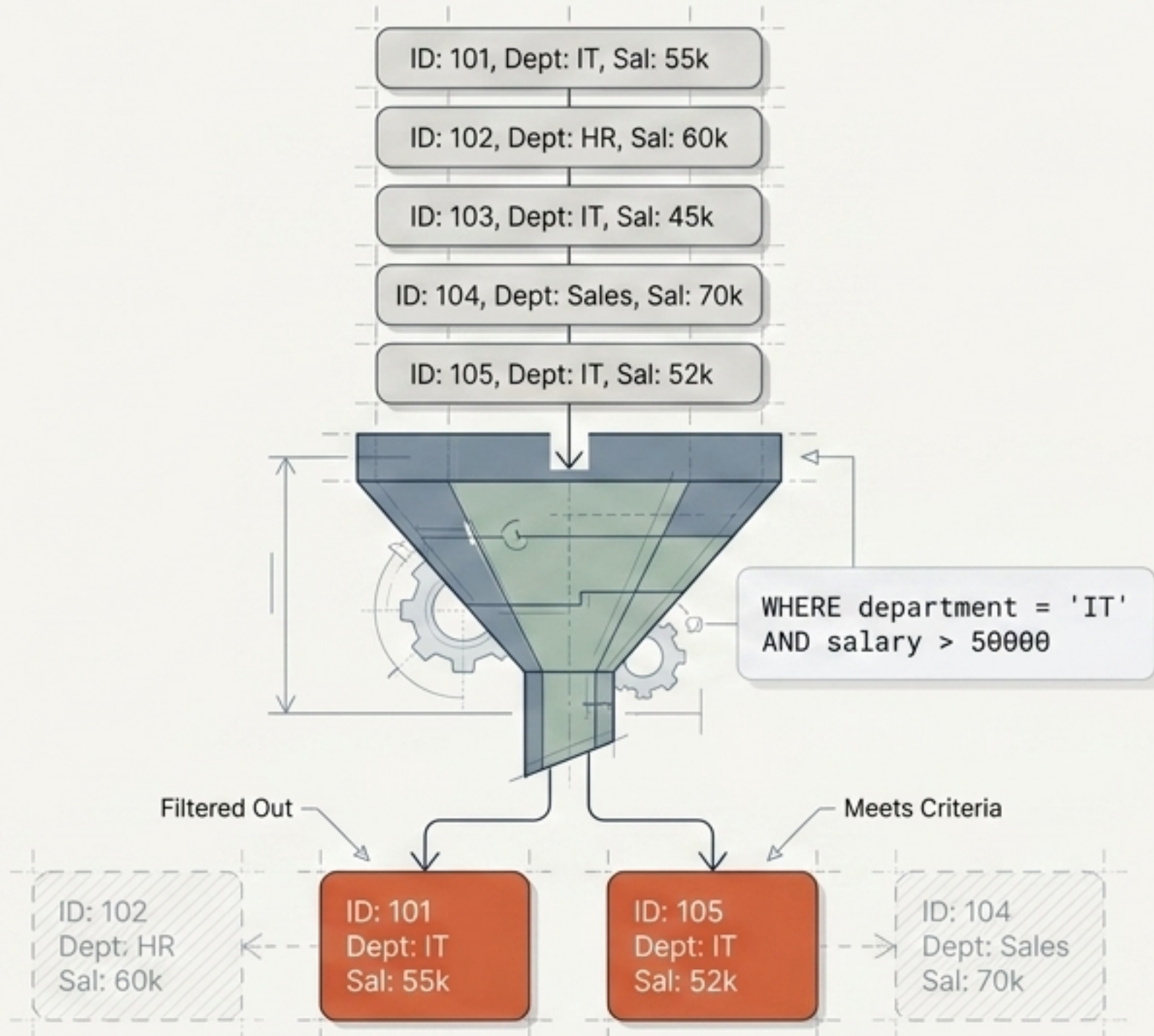
→ **SELECT** name, course **FROM** students **LIMIT** 5;

SELECT: What columns do we want to see? (Use * for all).

FROM: Which table are we pulling from?

LIMIT: How many rows should we return? (Crucial for large datasets).

Filtering the Noise



> / < / !=

Comparisons

IN ('IT', 'HR')

Multiple exact matches

BETWEEN 45000
AND 60000

Ranges

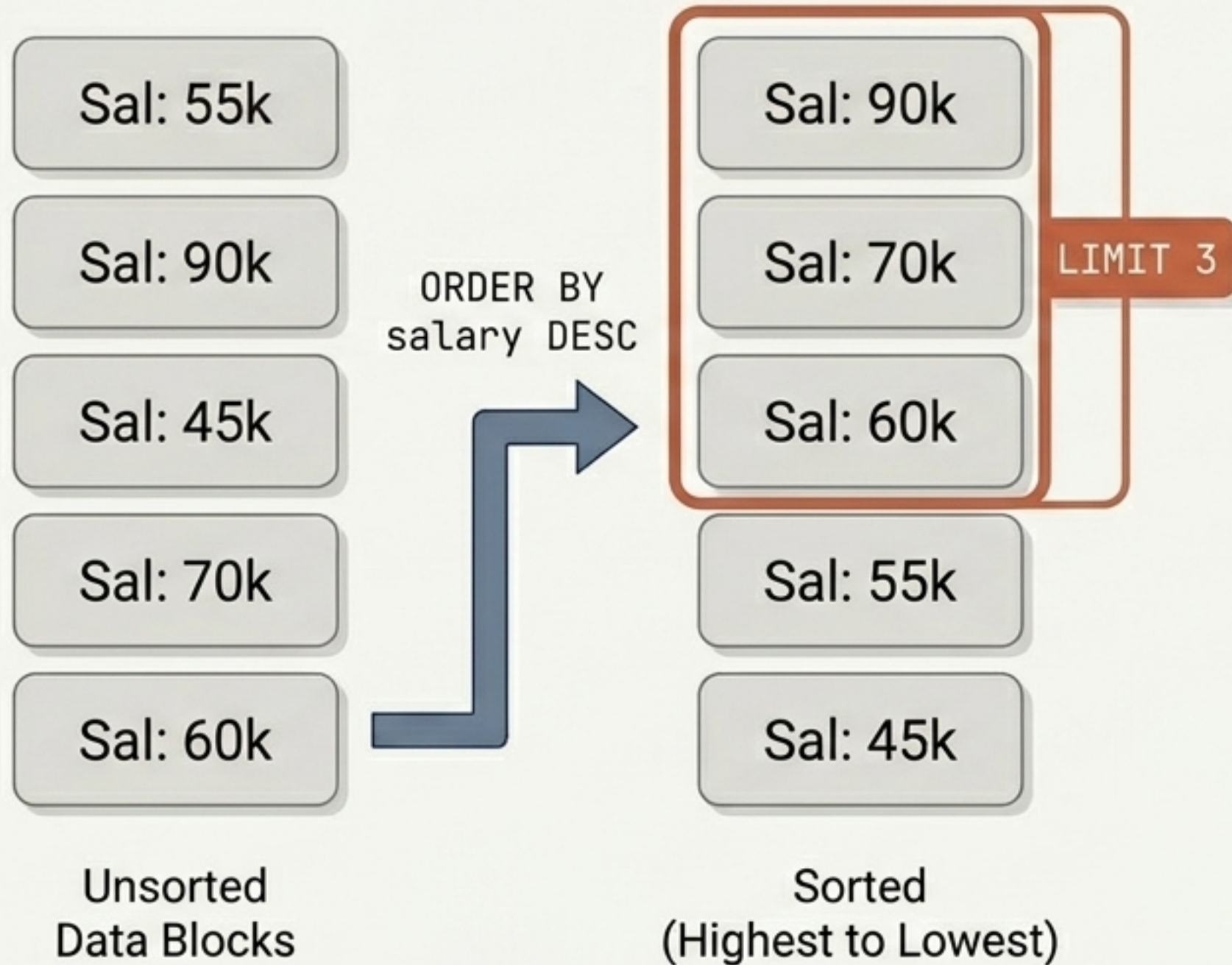
LIKE 'R%'

Pattern matching
(starts with R)

IS NULL

Find missing values

Sorting for Relevance



```
ORDER BY department ASC,  
salary DESC;
```

Multi-column sorting: First groups alphabetically by department, then ranks highest-to-lowest salary within each.

Pro Tip / Interview Alert:

Finding the 'Latest Order' or 'Most Expensive Product' always relies on combining `ORDER BY DESC` with `LIMIT 1`.

Phase 2: Summarization & Scale

From Rows to Metrics: Aggregate **functions** squash thousands of rows into a single, actionable business value.

Total Revenue
\$9,876,543
SUM()

Avg Order Value
\$450.75
AVG()

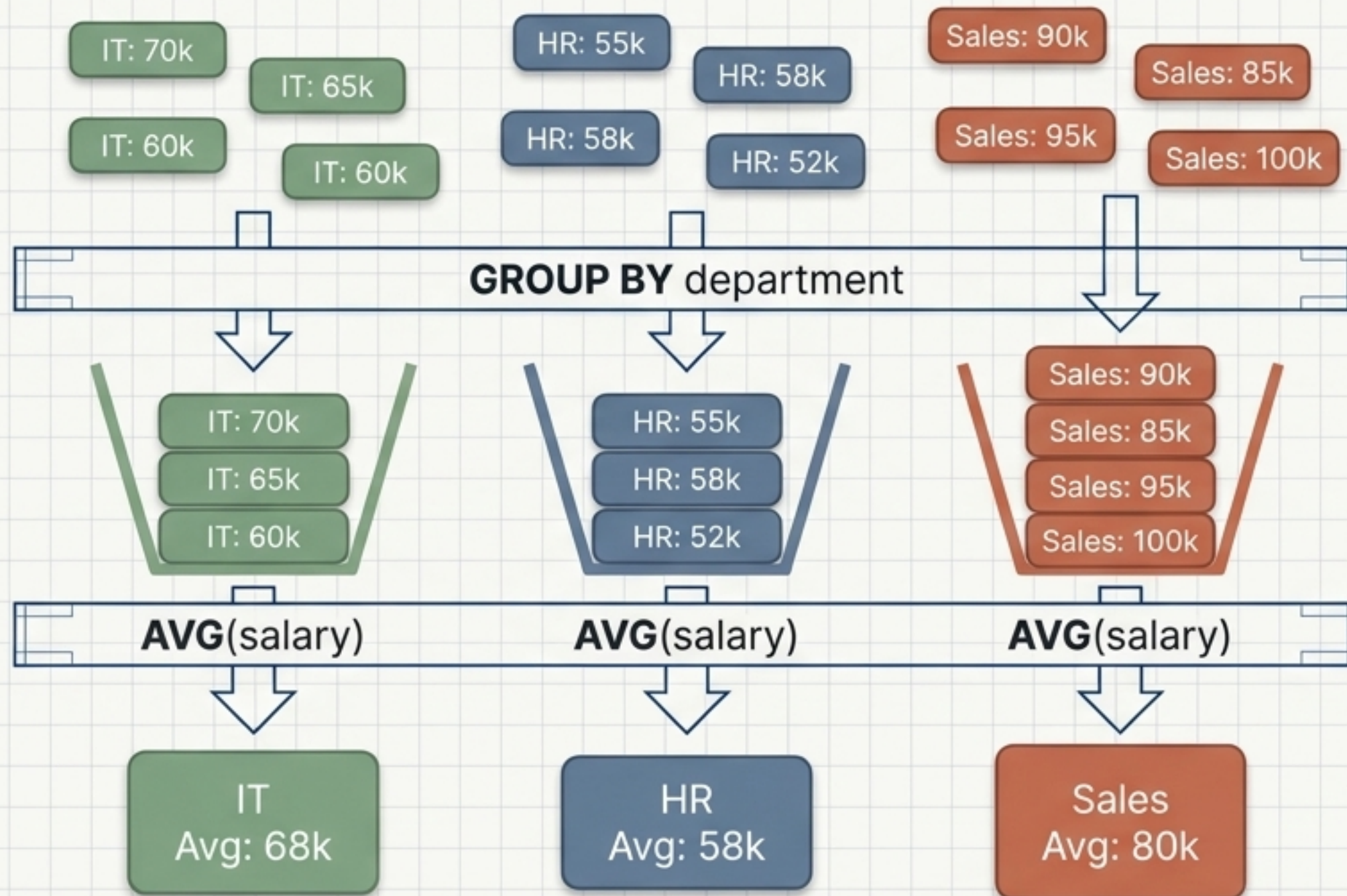
Total Customers
21,345
COUNT()

Max Purchase
\$15,000
MAX()

Min Purchase
\$25.99
MIN()

The Aggregation Funnel

Businesses rarely want raw rows. GROUP BY groups identical values, allowing aggregates to calculate a summary for each group.

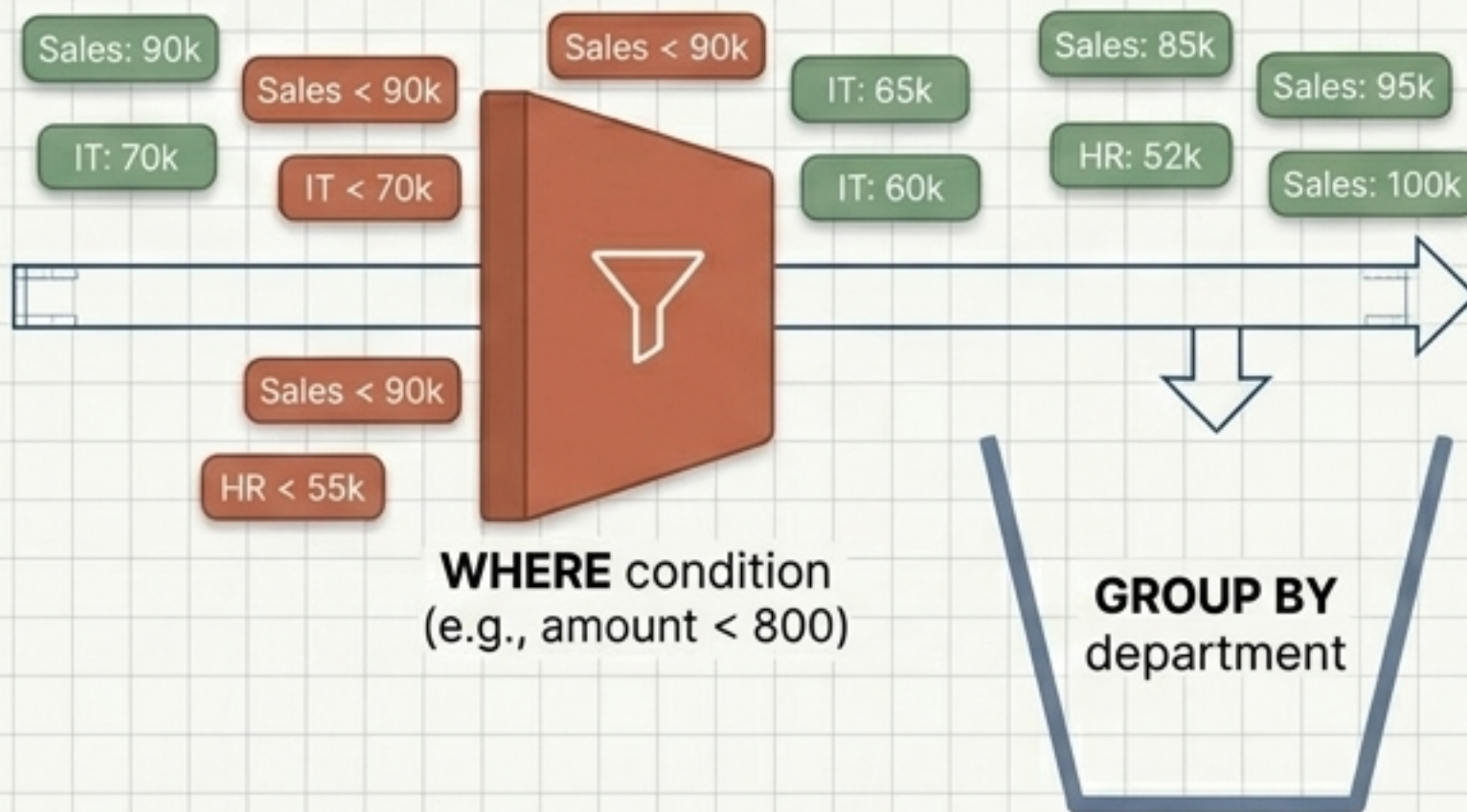


```
SELECT department,  
AVG(salary)  
FROM employees  
GROUP BY department;
```


Filtering Rows vs. Filtering Groups

WHERE

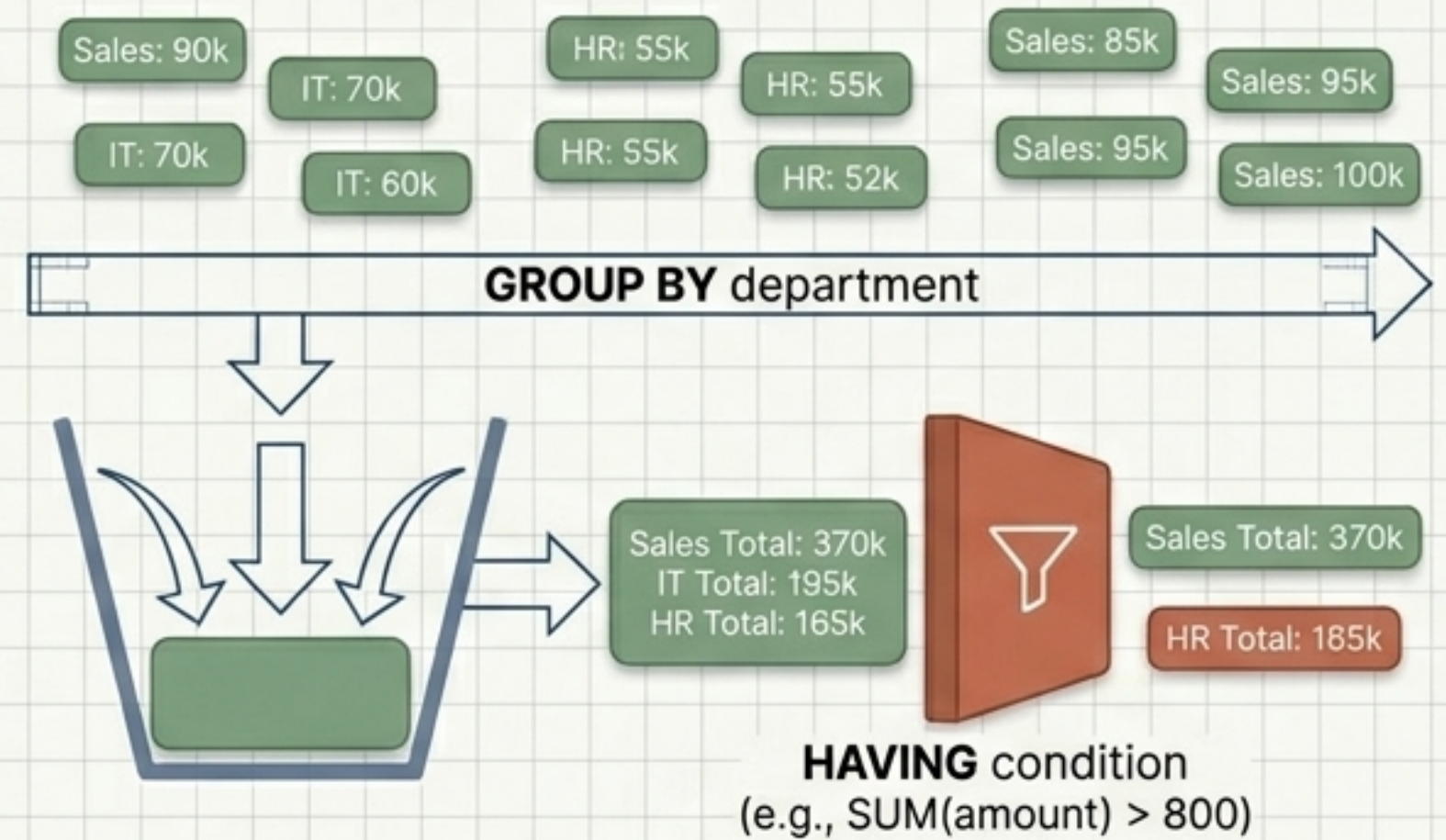
Filters raw rows



- Filters raw rows
- Executes before grouping
- Cannot use aggregate functions.

HAVING

Filters summarized groups



- Filters summarized groups
- Executes after grouping
- Used specifically for aggregate functions (e.g., HAVING SUM(amount) > 800).

Phase 3: The Relational Web

Connecting Siloed Data

Real-world databases split data into specialized tables to prevent duplication. JOIN is the mechanism that stitches them back together using a common column.



Which customer bought which product?

customers	
customer_id	name
101	'Alice'
102	'Bob'
103	'Charlie'

orders		
order_id	customer_id	amount
A1	101	500
A2	103	300
A3	102	800

JOIN

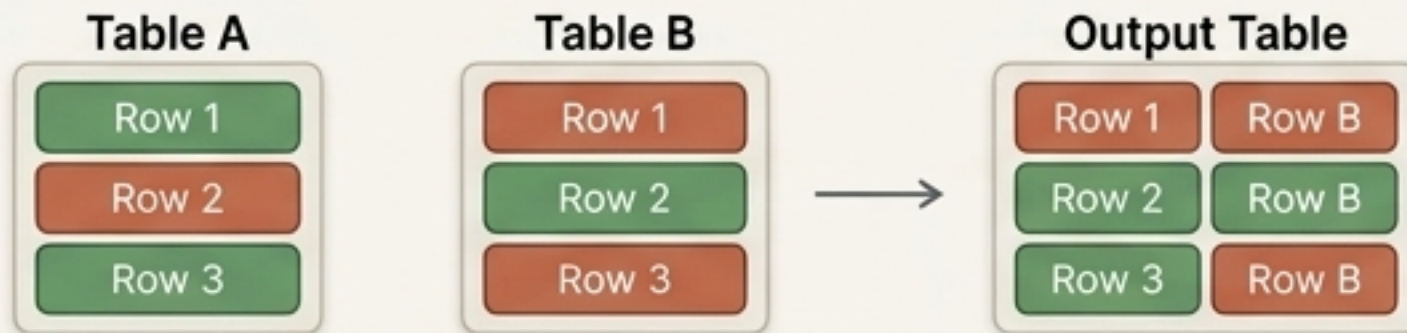
MERGED TABLE (customers JOIN orders)			
customer_id	name	order_id	amount
101	Alice	A1	500
101	Alice	A3	300
102	Bob	A2	800
103	Charlie	-	-

```
SELECT customers.name,  
       orders.amount  
FROM customers  
JOIN orders  
  ON customers.customer_id =  
     orders.customer_id;
```


The Great JOIN Matrix

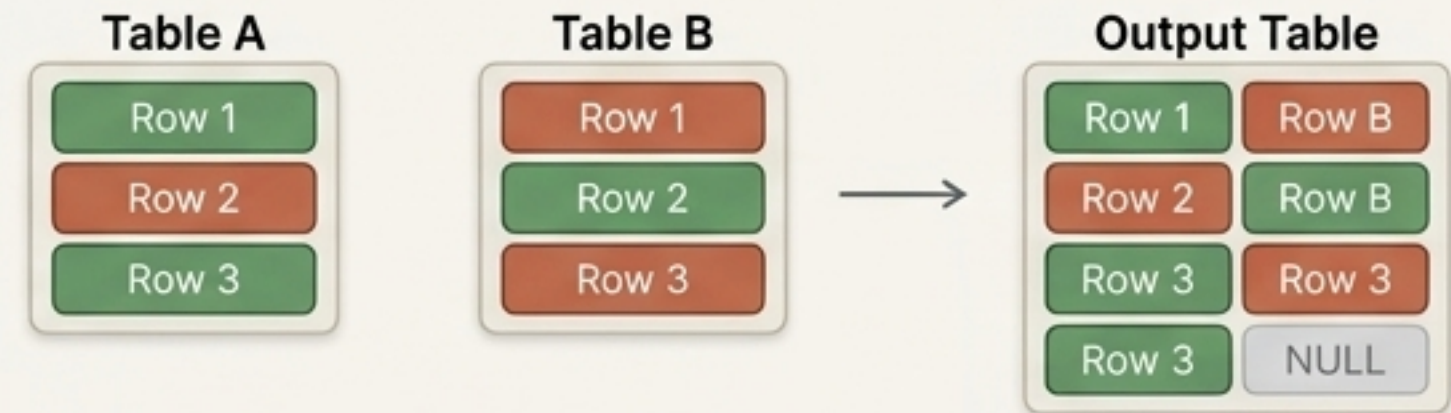
INNER JOIN

Only matching rows survive.



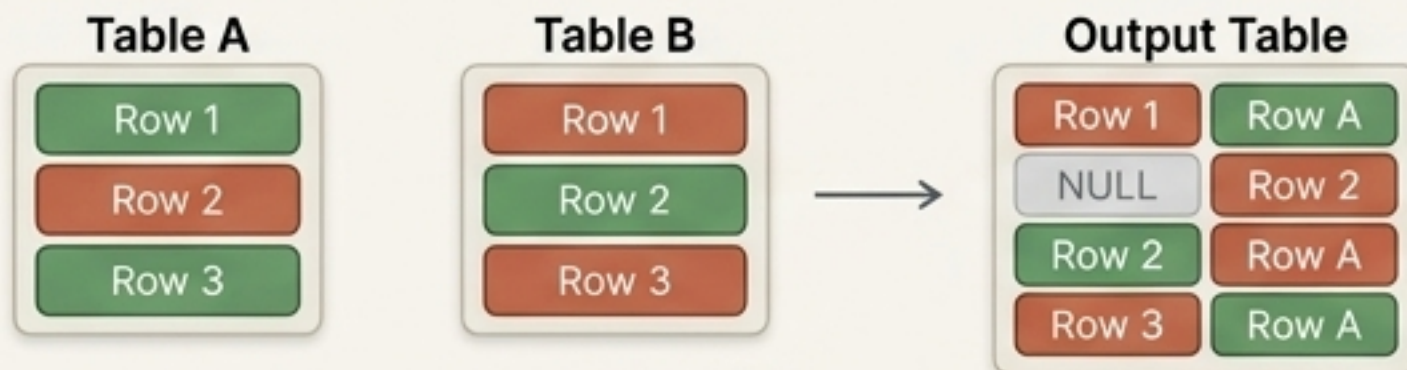
LEFT JOIN

All of Table A survives. Unmatched Table B values become NULL.



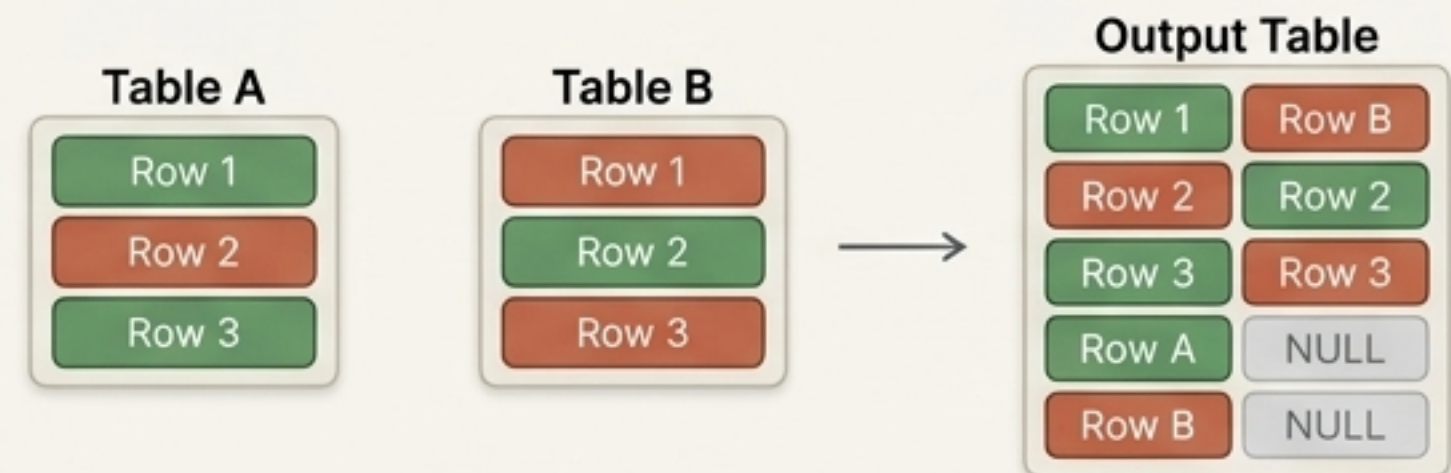
RIGHT JOIN

All of Table B survives. Unmatched Table A values become NULL.



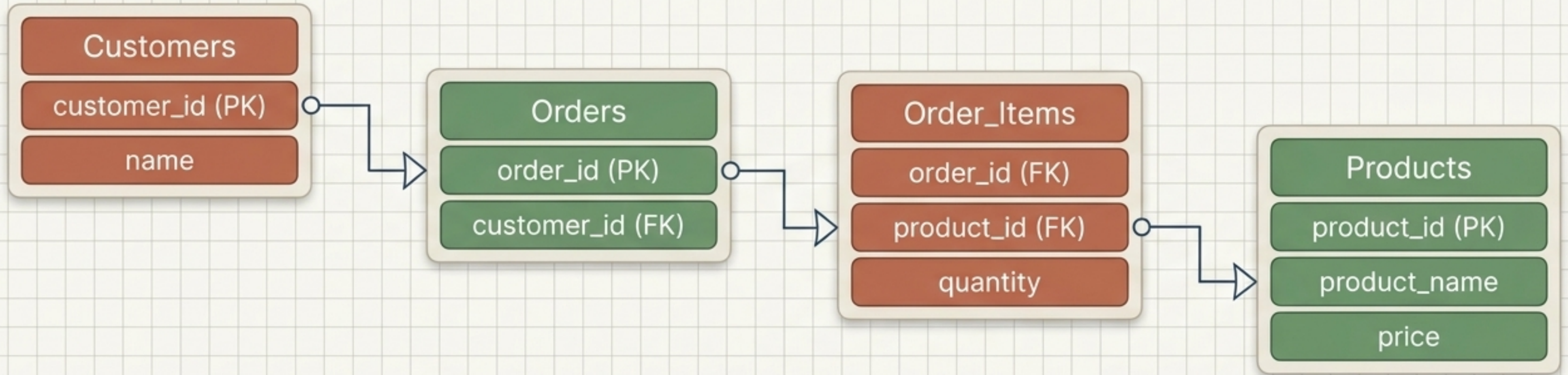
FULL JOIN

Everything survives; gaps filled with NULL.



Pro Tip: In Data Science, INNER JOIN (finding overlaps) and LEFT JOIN (keeping a primary dataset intact while appending data) cover 95% of use cases.

Multi-Table Architecture



```
SELECT c.name, p.product_name, o.order_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```



Self JOIN: Used for hierarchical data.

Phase 4: Advanced Architecture

The Subquery Strategy

Inner Query (Executes F):

Calculates the exact exact average.

WHERE salary >

(SELECT AVG(salary)
FROM employees)

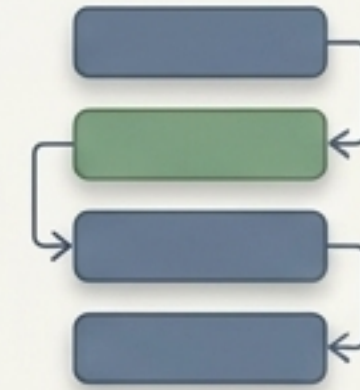
Outer Query (Executes

Second): Filters rows based on that average.

The Evolution of Syntax: Subqueries vs. CTEs



Subqueries (Nested)



CTEs (WITH)

Readability

Low (Reads inside-out)

High (Reads top-to-bottom)

Reusability

No (Must copy-paste)

Yes (Reference multiple times)

Debugging

Hard (Isolating logic is tough)

Easy (Run individual blocks)

Always prefer CTEs for complex, multi-step queries.

Building Pipelines with CTEs

CTEs allow you to break complex business logic **business logic into modular, digestible steps**—creating temporary tables that exist only for the duration of the query.

```
WITH total_orders AS (...)
```



```
, high_value AS (...)
```

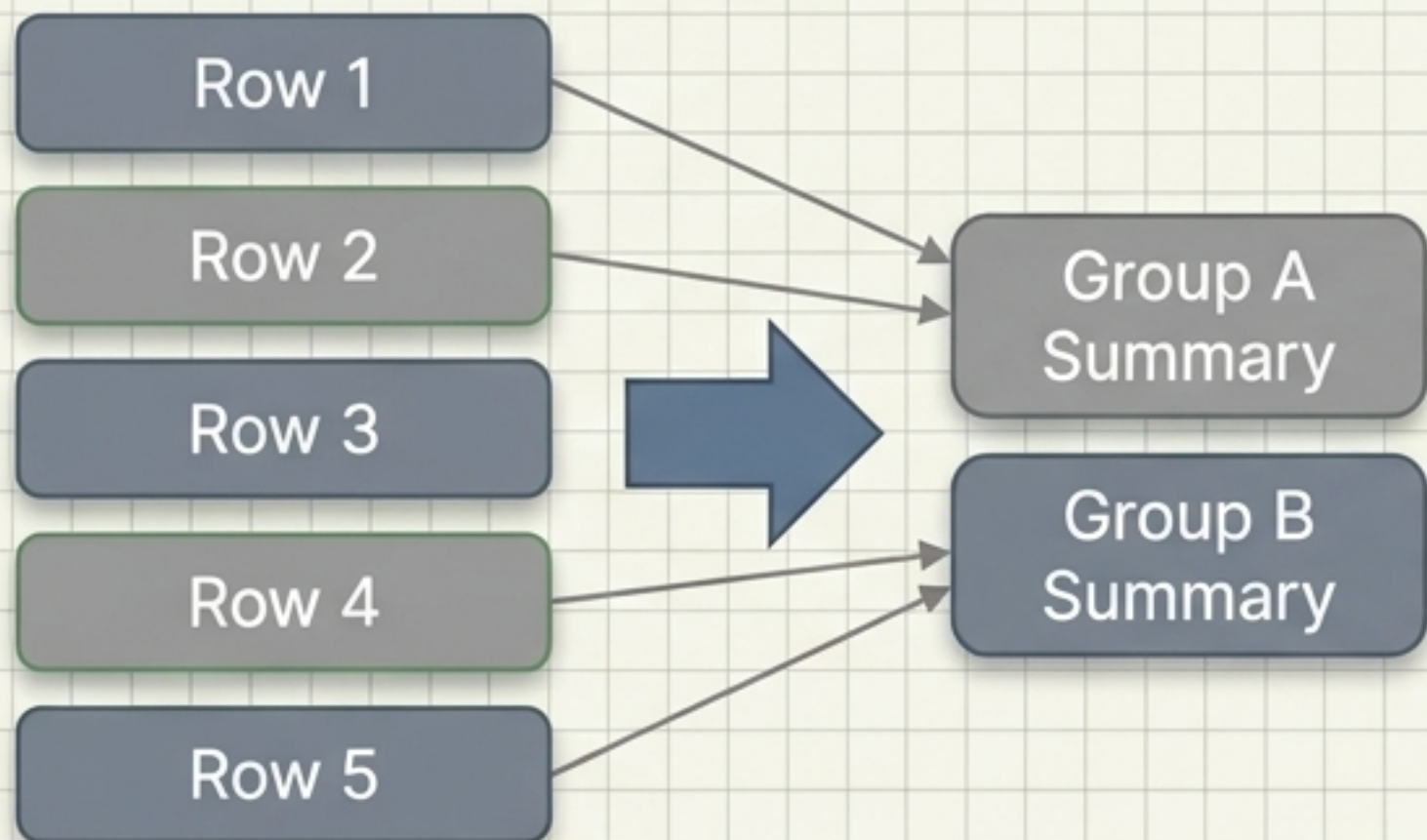


```
SELECT * FROM high_value;
```


The Power of Window Functions

GROUP BY reduces rows. **Window Functions** keep all rows intact while appending macro-insights.

GROUP BY



Reduces & Consolidates Rows.
Granularity is Lost.

Window Functions



Keeps All Rows Intact.
Appends Insights Alongside.

FUNCTION() OVER (PARTITION BY ... ORDER BY ...)

The Sliding Frame (PARTITION BY)

PARTITION BY divides the data into independent groups. The window function calculates its value within that frame, then resets when it hits a new partition.

Use Case: Top 3 salaries per department.

Rank calculation...

IT Department

IT Department

IT Department

HR Department

HR Department

HR Department

Emp ID: 101, Salary: \$120k, Rank: 1

Emp ID: 102, Salary: \$115k, Rank: 2

Emp ID: 103, Salary: \$110k, Rank: 3

Emp ID: 201, Salary: \$95k, Rank: 1

Emp ID: 202, Salary: \$90k, Rank: 2

Emp ID: 203, Salary: \$85k, Rank: 3

Independent 'IT'
Partition Frame.
Calculation is isolated
here.

Frame moves to next partition.
Calculation resets.

Calculation resets for
the 'HR' Partition.

```
SELECT emp_id, department, salary,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_within_dept  
FROM employees;
```


The Ranking Matrix

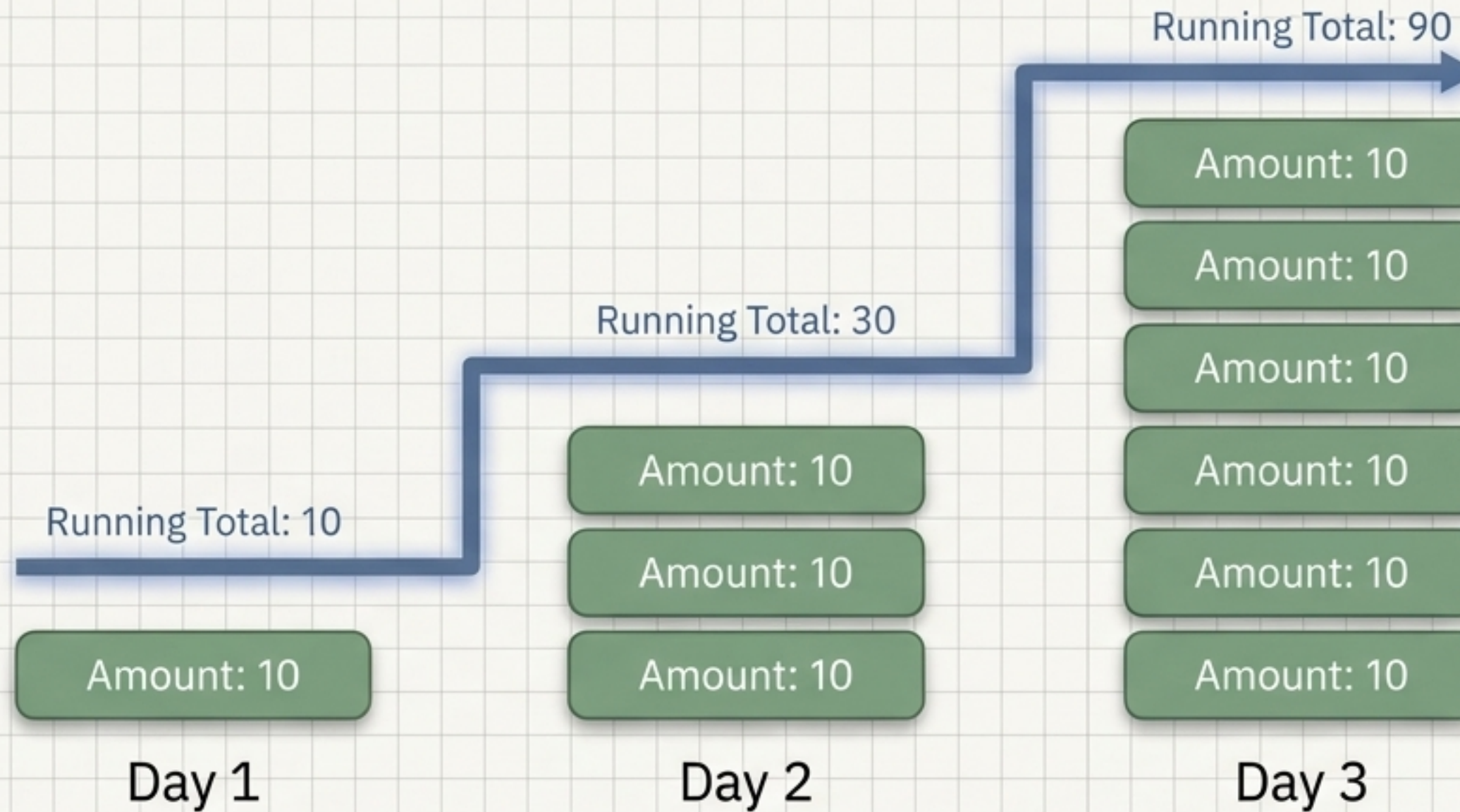
Raw Data	ROW_NUMBER()	RANK()	DENSE_RANK()
60k	1	1	1
55k	2	2	2
55k	3	2	2
50k	4	4	3

The diagram shows two rows of data with identical values (55k) in the 'Raw Data' column. In the 'RANK()' column, the first 55k is ranked 2 and the second 55k is also ranked 2, with a red circle and an arrow pointing to the second row labeled 'Skip for Ties'. In the 'DENSE_RANK()' column, the first 55k is ranked 2 and the second 55k is also ranked 2, with a red circle and an arrow pointing to the second row labeled 'No Gap'.

Pro Tip / Interview Alert:

Use `DENSE_RANK()` when asked to find the 'Top N' values (like the 2nd highest salary) to ensure ties don't accidentally skip the rank you are looking for.

Time Series & Cumulative Insights

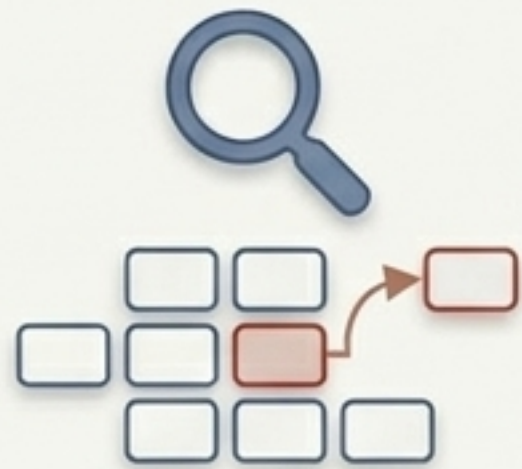


By omitting PARTITION BY and using only ORDER BY inside the window, the engine aggregates cumulatively row-by-row, creating running totals and moving averages.

```
SUM(amount) OVER (ORDER BY order_date) AS running_total
```


The Analytics Playbook

Advanced SQL is about pattern recognition. These **four architectural patterns solve 90% of technical interview challenges**.



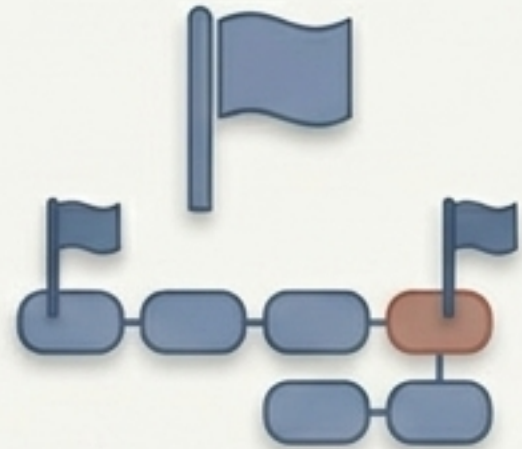
Finding Missing Data
(Customers with zero orders)

LEFT JOIN + IS NULL



Comparing to Averages
(Above-average spenders)

CTE + JOIN



First/Last Actions
(First purchase per user)

ROW_NUMBER() +
PARTITION BY

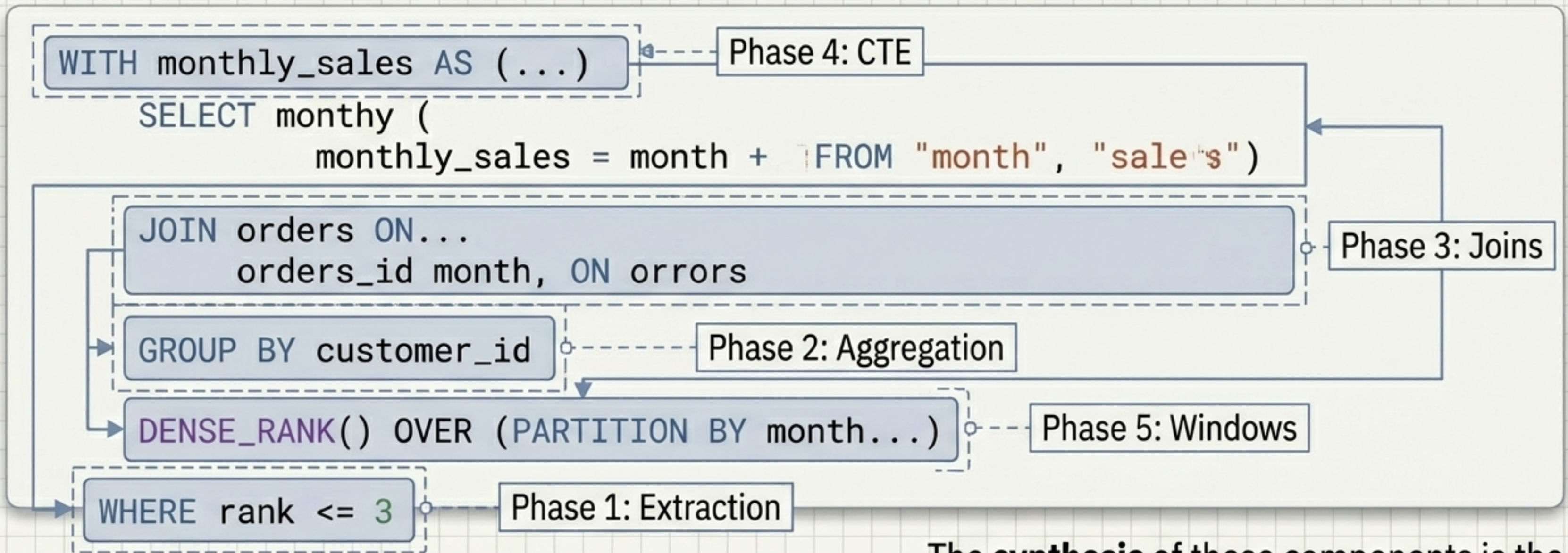


Detecting Duplicates

ROW_NUMBER() > 1

The Anatomy of a Production Query

Advanced SQL is not a new language. It is simply the basic building blocks—**Extraction, Summarization, Relational Joins, and Windows**—stacked together logically to engineer **intelligence**.



The **synthesis** of these components is the **engine of modern data intelligence**.